

Noah AI Assistant — PayTo Flutter & Offerhopper MCP Integration Plan

1. Overview & Architecture

Noah functions as the embedded conversational intelligence inside the PayTo Flutter app. Noah interprets natural language requests, invokes PayTo application workflows, retrieves grounded facts from documents and the backend, and displays interactive UI elements (barcodes, offer carousels, store split breakdowns, maps).

For multi-item grocery basket optimization and route planning, Noah integrates directly with **Offerhopper MCP** (<https://mcp.offerhopper.ai/mcp>).

End-to-End System Flow



[NoahActionDispatcher (Flutter)] → Renders In-Chat Dynamic UI [Components](#):

- └ response_mode == "functional" → High-contrast Barcode Modal with brightness boost
- └ response_mode == "hybrid" → Offer Carousels / Product Comparison Tiles
- └ response_mode == "hybrid" → Offerhopper Multi-Store Savings & Route Card
- └ response_mode == "navigation" → Launches Maps / Directions
- └ response_mode == "text" → Conversational Response Bubble

2. Offerhopper MCP Specification & Protocol

According to the official [Offerhopper MCP Developer Guide](#), the connection and tool calling protocol is defined as follows:

Connection Parameters

- **MCP Server Endpoint:** `https://mcp.offerhopper.ai/mcp`
- **Transport:** Streamable HTTP / JSON-RPC 2.0
- **Authentication:** None (No API key required)
- **Covered Retailers in Germany:** Aldi, Lidl, Rewe, Edeka, Penny, Norma, Netto, DM, Rossmann, Müller

Tool Definition: `plan_optimal_shopping_route`

Parameter	Type	Required	Description	Example
<code>items</code>	<code>string</code>	✓	Natural language shopping list	"milk, eggs, bread, butter"
<code>location</code>	<code>string</code>	✓	German city, address, or 5-digit PLZ	"Hamburg", "80331", "Berlin Mitte"
<code>end_location</code>	<code>string</code>	✗	Optional destination for A → B commute	"Alexanderplatz, Berlin"
<code>travel_mode</code>	<code>string</code>	✗	"car", "bicycle", or "pedestrian" (default: "car")	"bicycle"
<code>max_radius_km</code>	<code>number</code>	✗	Maximum search radius in	5.0

Parameter	Type	Required	Description	Example
			kilometers	
max_stores	integer	✗	Max number of store stops in route	3
hour_cost	number	✗	Time penalty value in EUR/hour (default: 12.0)	6.0 (frugal) / 25.0 (fast)
km_cost	number	✗	Travel cost in EUR/km (0 for bike/walk)	0.15
shopping_time_per_store	integer	✗	Estimated minutes spent per store stop	10

Expected Output Payload from Offerhopper MCP

```
{
  "total_basket_cost": 8.45,
  "single_store_cheapest": 11.20,
  "total_savings_eur": 2.75,
  "travel_penalty_eur": 0.60,
  "net_savings_eur": 2.15,
  "travel_mode": "bicycle",
  "total_distance_km": 3.2,
  "total_duration_min": 24,
  "share_url": "https://offerhopper.ai/r/a8f9c12e",
  "stops": [
    {
      "stop_number": 1,
      "merchant": "Lidl",
      "address": "Stresemannstraße 120, Hamburg",
      "items": [
        {"name": "Whole Milk 3.5%", "price": 0.99, "quantity": 1},
        {"name": "Free-Range Eggs 10pk", "price": 1.89, "quantity": 1}
      ],
      "subtotal_eur": 2.88
    },
    {
      "stop_number": 2,
```

```

    "merchant": "Aldi",
    "address": "Max-Brauer-Allee 45, Hamburg",
    "items": [
        {"name": "Organic Sourdough Bread", "price": 1.49, "quantity": 1},
        {"name": "German Butter 250g", "price": 1.79, "quantity": 1}
    ],
    "subtotal_eur": 3.28
}
]
}

```

3. Python Backend MCP Integration (noah_backend/app/tools/offerhopper.py)

```

"""Offerhopper MCP Client for multi-item grocery basket and route optimization."""

import logging
from typing import Any, Dict, Optional
import httpx

logger = logging.getLogger(__name__)

OFFERHOPPER_MCP_URL = "https://mcp.offerhopper.ai/mcp"

def call_offerhopper_mcp(
    items: str,
    location: str,
    travel_mode: str = "car",
    max_radius_km: Optional[float] = None,
    hour_cost: float = 12.0,
) -> Dict[str, Any]:
    """Execute plan_optimal_shopping_route on the live Offerhopper MCP server."""
    payload = {
        "jsonrpc": "2.0",
        "id": 1,
        "method": "tools/call",
        "params": {
            "name": "plan_optimal_shopping_route",
            "arguments": {
                "items": items,
                "location": location,
                "travel_mode": travel_mode,
                "hour_cost": hour_cost,
                **({"max_radius_km": max_radius_km} if max_radius_km else {}),
            },
        },
    },

```

```

}

try:
    with httpx.Client(timeout=30.0) as client:
        # Initialize session for streamable HTTP transport
        init_resp = client.post(
            OFFERHOPPER_MCP_URL,
            json={
                "jsonrpc": "2.0",
                "id": 0,
                "method": "initialize",
                "params": {
                    "protocolVersion": "2025-03-26",
                    "clientInfo": {"name": "payto-noah-backend", "version": "1.0"},
                },
            },
        )
        session_id = init_resp.headers.get("mcp-session-id")
        headers = {"mcp-session-id": session_id} if session_id else {}

        resp = client.post(OFFERHOPPER_MCP_URL, json=payload, headers=headers)
        resp.raise_for_status()
        data = resp.json()

        if "result" in data:
            return data["result"]
        elif "error" in data:
            logger.error(f"Offerhopper MCP error: {data['error']}")
except Exception as e:
    logger.exception(f"Failed to communicate with Offerhopper MCP: {e}")

return {}

```

4. Flutter File Structure & Implementation

```

lib/
├── models/
│   ├── noah_chat_models.dart      # Noah API request/response Dart models
│   └── offerhopper_models.dart    # Offerhopper store breakdown & route models
├── services/
│   ├── noah_api_service.dart      # Connects to FastAPI /api/chat
│   └── noah_action_dispatcher.dart # Routes actions to Barcode, Offers, Maps, Routes
├── providers/
│   └── noah_chat_provider.dart     # Multi-turn conversation state & active context
└── screens/

```

	└─ noah_chat_screen.dart	# Floating assistant sheet / Fullscreen chat
	└─ widgets/	
	└─ noah/	
	└─ noah_message_bubble.dart	# Chat text bubble with sender badge
	└─ noah_barcode_card.dart	# High-contrast 1D/QR barcode with brightness boost
	└─ noah_offer_carousel.dart	# Horizontal scroll of active deals & discounts
	└─ noah_product_tile.dart	# Product price comparison card with add-to-list
	└─ noah_route_card.dart	# Offerhopper multi-store route & savings card

5. Flutter Code Implementations

A. Offerhopper Model (lib/models/offerhopper_models.dart)

```
class OfferhopperItem {
  final String name;
  final double price;
  final int quantity;

  OfferhopperItem({required this.name, required this.price, this.quantity = 1});

  factory OfferhopperItem.fromJson(Map<String, dynamic> json) {
    return OfferhopperItem(
      name: json['name'] ?? '',
      price: (json['price'] as num?).toDouble() ?? 0.0,
      quantity: json['quantity'] ?? 1,
    );
  }
}

class OfferhopperStop {
  final int stopNumber;
  final String merchant;
  final String address;
  final List<OfferhopperItem> items;
  final double subtotalEur;

  OfferhopperStop({
    required this.stopNumber,
    required this.merchant,
    required this.address,
    required this.items,
    required this.subtotalEur,
  });

  factory OfferhopperStop.fromJson(Map<String, dynamic> json) {
```

```

return OfferhopperStop(
    stopNumber: json['stop_number'] ?? 1,
    merchant: json['merchant'] ?? '',
    address: json['address'] ?? '',
    items: (json['items'] as List? ?? [])
        .map((i) => OfferhopperItem.fromJson(i))
        .toList(),
    subtotalEur: (json['subtotal_eur'] as num?).toDouble() ?? 0.0,
);
}
}

class OfferhopperRouteResult {
    final double totalBasketCost;
    final double totalSavingsEur;
    final double netSavingsEur;
    final String travelMode;
    final double totalDistanceKm;
    final int totalDurationMin;
    final String shareUrl;
    final List<OfferhopperStop> stops;

    OfferhopperRouteResult({
        required this.totalBasketCost,
        required this.totalSavingsEur,
        required this.netSavingsEur,
        required this.travelMode,
        required this.totalDistanceKm,
        required this.totalDurationMin,
        required this.shareUrl,
        required this.stops,
    });

    factory OfferhopperRouteResult.fromJson(Map<String, dynamic> json) {
        return OfferhopperRouteResult(
            totalBasketCost: (json['total_basket_cost'] as num?).toDouble() ?? 0.0,
            totalSavingsEur: (json['total_savings_eur'] as num?).toDouble() ?? 0.0,
            netSavingsEur: (json['net_savings_eur'] as num?).toDouble() ?? 0.0,
            travelMode: json['travel_mode'] ?? 'car',
            totalDistanceKm: (json['total_distance_km'] as num?).toDouble() ?? 0.0,
            totalDurationMin: json['total_duration_min'] ?? 0,
            shareUrl: json['share_url'] ?? '',
            stops: (json['stops'] as List? ?? [])
                .map((s) => OfferhopperStop.fromJson(s))
                .toList(),
        );
    }
}

```

B. Noah Action Dispatcher (lib/services/noah_action_dispatcher.dart)

```
import 'package:flutter/material.dart';
import 'package:url_launcher/url_launcher.dart';
import '../models/noah_chat_models.dart';
import '../widgets/noah/noah_barcode_card.dart';
import '../widgets/noah/noah_route_card.dart';

class NoahActionDispatcher {
  static void dispatch(BuildContext context, NoahResponse response) {
    final actions = response.plannerActions;
    final entities = response.entities;

    for (final action in actions) {
      switch (action) {
        case 'DISPLAY_BARCODE':
        case 'OPEN_WALLET_CARD':
          showModalBottomSheet(
            context: context,
            isScrollControlled: true,
            backgroundColor: Colors.transparent,
            builder: (ctx) => NoahBarcodeCard(
              cardName: entities.loyaltyCard ?? entities.brand ?? 'PAYBACK',
              cardNumber: '9283 4810 2938 12', // Populated from user's wallet
            ),
          );
          break;

        case 'FIND_CHEAPEST_BASKET':
        case 'OPTIMIZE_SHOPPING_ROUTE':
        case 'SPLIT_BASKET_ACROSS_MERCHANTS':
          if (response.offerhopperData != null) {
            showModalBottomSheet(
              context: context,
              isScrollControlled: true,
              backgroundColor: Colors.transparent,
              builder: (ctx) => NoahRouteCard(
                routeResult: response.offerhopperData!,
              ),
            );
          }
          break;

        case 'PLAN_ROUTE':
        case 'OPEN_GOOGLE_MAPS':
          final query = entities.merchant ?? entities.location ?? '';
          _launchMapsUrl(query);
          break;
      }
    }
  }
}
```

```

}

static Future<void> _launchMapUrl(String query) async {
  final url = Uri.parse('https://www.google.com/maps/search/?api=1&query=${Uri.encodeComponent(query)}');
  if (await canLaunchUrl(url)) {
    await launchUrl(url, mode: LaunchMode.externalApplication);
  }
}
}
}

```

C. Offerhopper Route & Savings Card Widget (lib/widgets/noah/noah_route_card.dart)

```

import 'package:flutter/material.dart';
import 'package:url_launcher/url_launcher.dart';
import '../models/offerhopper_models.dart';

class NoahRouteCard extends StatelessWidget {
  final OfferhopperRouteResult routeResult;

  const NoahRouteCard({Key? key, required this.routeResult}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);

    return Container(
      margin: const EdgeInsets.symmetric(vertical: 8, horizontal: 12),
      padding: const EdgeInsets.all(16),
      decoration: BoxDecoration(
        color: theme.cardColor,
        borderRadius: BorderRadius.circular(16),
        boxShadow: [
          BoxShadow(
            color: Colors.black.withOpacity(0.08),
            blurRadius: 10,
            offset: const Offset(0, 4),
          ),
        ],
      ),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          // Header: Total & Savings Badge
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceBetween,
            children: [
              Column(

```

```

        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            'Optimized Basket Total',
            style: theme.textTheme.labelMedium?.copyWith(color: Colors.grey),
          ),
          Text(
            '€${routeResult.totalBasketCost.toStringAsFixed(2)}',
            style: theme.textTheme.headlineSmall?.copyWith(fontWeight: FontWeight.bold),
          ),
        ],
      ),
      Container(
        padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 6),
        decoration: BoxDecoration(
          color: Colors.green.withOpacity(0.12),
          borderRadius: BorderRadius.circular(20),
        ),
        child: Text(
          'Save €${routeResult.totalSavingsEur.toStringAsFixed(2)}',
          style: const TextStyle(
            color: Colors.green,
            fontWeight: FontWeight.bold,
            fontSize: 13,
          ),
        ),
      ),
    ],
  ),
  const SizedBox(height: 12),
  const Divider(height: 1),
  const SizedBox(height: 12),

  // Route Stops
  Text('Shopping Stops (${routeResult.stops.length})', style: theme.textTheme.titleSmall),
  const SizedBox(height: 8),
  ...routeResult.stops.map((stop) => Padding(
    padding: const EdgeInsets.only(bottom: 10),
    child: Row(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        CircleAvatar(
          radius: 12,
          backgroundColor: theme.primaryColor,
          child: Text(
            '${stop.stopNumber}',
            style: const TextStyle(fontSize: 12, color: Colors.white, fontWeight: FontWe:
          ),
        ),
        const SizedBox(width: 10),
        Expanded(

```

```

        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              stop.merchant,
              style: const TextStyle(fontWeight: FontWeight.bold, fontSize: 14),
            ),
            Text(
              stop.items.map((i) => '${i.quantity}x ${i.name} (€${i.price.toStringAsFixed(2)})',
              style: theme.textTheme.bodySmall?.copyWith(color: Colors.grey[700])),
            ),
          ],
        ),
        Text(
          '€${stop.subtotalEur.toStringAsFixed(2)}',
          style: const TextStyle(fontWeight: FontWeight.bold, fontSize: 13),
        ),
      ],
    ),
  ),
),
),
const SizedBox(height: 10),

// Open Interactive Route Map Button
if (routeResult.shareUrl.isNotEmpty)
  SizedBox(
    width: double.infinity,
    child: ElevatedButton.icon(
      onPressed: () async {
        final uri = Uri.parse(routeResult.shareUrl);
        if (await canLaunchUrl(uri)) {
          await launchUrl(uri, mode: LaunchMode.externalApplication);
        }
      },
      icon: const Icon(Icons.map_outlined, size: 18),
      label: const Text('Open Interactive Route Map'),
      style: ElevatedButton.styleFrom(
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(10)),
      ),
    ),
  ),
),
),
],
),
);
}
}

```

6. Verification & Test Scenarios

1. Offerhopper Multi-Item Basket:

- *Input:* "Find the cheapest basket for milk, eggs, bread, butter in Hamburg."
- *Result:* Backend invokes Offerhopper `plan_optimal_shopping_route`. Flutter renders `NoahRouteCard` with Aldi + Lidl item split, subtotal prices, and "Open Interactive Route Map" link.

2. Loyalty Barcode Display:

- *Input:* "Show me my Payback barcode."
- *Result:* Flutter opens high-contrast barcode modal with screen brightness boost.

3. Compound Offer + Payment Action:

- *Input:* "Find ice cream deals at Netto below €3, then show my Payback card."
- *Result:* Flutter displays horizontal offer carousel and pops the barcode widget ready for checkout.

4. PayTo RAG Knowledge:

- *Input:* "Does Payto store my payment details?"
- *Result:* Noah replies with document-grounded answer: *"No, PayTo never stores your payment details, transaction history, or builds a profile."*